Databases Project
2025/2026

(2026/03/05: versão 1.3 do enunciado; fixed ponderações)
(2026/03/05: versão 1.2 do enunciado; fixed data entrega intermédia)
(2026/03/05: versão 1.1 do enunciado; fixed typo na ponderação da defesa final)
(2026/03/03: versão 1.0 do enunciado)

Introduction

The goal of this project is to provide students with practical experience in designing and implementing a database-centric system, exposed through a REST API and supported by a relational DBMS.

Objectives

After completing this project, students should be able to:

- Understand how a database application project is organized, planned, and executed.
- Design conceptual and physical data models for a real-world mobility scenario.
- Implement and test a transactional system using PostgreSQL
- Develop and expose functionalities through a REST API in Python.
- Install, configure, manage, and tune a modern relational DBMS
- Understand client and server-side programming.

Groups

The project is to be done in **groups of 3** students.
IMPORTANT:

- The members of each group **must** be enrolled in PL classes with the same Professor.
- Group defenses are **individual** (each member is evaluated individually).

Quality attributes for a good project

Your application must include:

(a) A transactional relational DBMS (**PostgreSQL**).
(b) Data modelling using **ONDA**.
(c) A distributed architecture with **REST API**.
(d) SQL functionalities on the server side.
(e) Explicit handling of transactions and potential concurrency conflicts.
(f) Proper authentication/authorization strategy.
(g) Error avoidance, detection, and mitigation mechanisms.
(h) Good project documentation and reproducible setup.

Your application must also respect the functional requirements defined in **Annex A** and execute without "visible problems" or "crashes". To fulfill the objectives of this assignment, you can be as creative as you want, provided you implement the list of required features.

Do not start coding right away - take time to think about the problem and structure your development plan and design.

Milestones and deliverables

1. **Midterm delivery (30% of the grade) –** 23:55 of March 27th

The following artifacts must be uploaded at Inforestudante until the deadline (each group must select a member for performing this task):

- **Report** with the following information:
  - Team members and contacts
  - Brief description of the project
  - Detailed definition of transactions, potential concurrency conflicts (those that are not directly handled by the DBMS), and the strategy to avoid them
  - Development plan: planned tasks, initial work division per team member, timeline
- **ER diagram**
  - Use ONDA to design the ER, and include a description of entities, attributes, integrity rules, etc.
- **Relational data model**
  - Use ONDA to output the physical model of the database (i.e., the tables)
- **The ONDA project, exported as a JSON file.**

Intermediate Defense – April 6th to April 10th

The group must sign up for an available time slot for the defense in Inforestudante, before the delivery due date (the list of available slots will be released before the deadline). **All students must be present, but the defense is individual.**

2. **Quick checkpoint 1 (5% of the grade) –** During PL class on the 12th week (week of 27th of April); everyone's presence in class is mandatory, slots will be made available in Inforestudante
   - **REST API** (provided in the demo) working, connected to the project DB, with a basic retrieval endpoint (available in the demo)
   - **Authentication** functionality implemented (login endpoint & authorization example in a separate endpoint)

3. **Final delivery (65% of the grade) –** 23:55 of May 21st

The project outcomes must be submitted to Inforestudante by the deadline. Each group must select a member for performing this task.

- **Final report** with (the following items in a single document):
  - **Installation manual** describing how to deploy and run the software you developed
  - **User manual** describing the submitted Postman collection requests to test the application
  - **Final ER and relational data models (ONDA jpeg outputs)**
  - **Execution report:** make sure you specify which tasks were done by each team member and the effort involved (e.g., hours)
  - **All the information, details, and design decisions you consider relevant to understand how the application was built and how it satisfies the requirements of the project (do not make generic descriptions of the endpoints, include only relevant information regarding your process and implementation)**
- **Video demo of project (Max. 5-min)**
- **Source code and Scripts:**
  - Include the source code, scripts, executable files, and libraries necessary for compiling and running the software
  - Complete DB creation scripts
  - Postman collection with all the requests required to test the application
  - The ONDA project, exported as a JSON file.

Final Defense – June 1st to June 5th

- The group must sign up for any available time slot for the defense in Inforestudante, before the delivery due date (the list of available slots will be released before the deadline for the final delivery)
- **All students must be present, but the defense will be <u>individual</u>, ordered by name.**

Academic integrity and AI disclosure

Plagiarism and fraud are strictly prohibited and will not be tolerated. The use of AI-based tools (e.g., ChatGPT, Gemini, Claude, or similar systems) is permitted only with explicit disclosure and solely to support the learning process, not to generate solutions. Students must clearly state how AI tools were used (e.g., writing assistance, code suggestions, documentation refinement, brainstorming). Content generated by AI systems does not qualify as student-produced work, and any use of such tools must be explicitly declared in the submission. Failure to disclose AI use will be treated as academic misconduct, resulting in a grade of 0 and possible disciplinary action. Additionally, submissions that clearly correspond to solutions produced by large language models (LLMs) will not be accepted as valid student work, regardless of the student's ability to explain them during evaluation.

Assessment

- This project accounts for 7 points (out of 20) of the total grade in the Databases course
- Midterm delivery corresponds to 30% of the grade of the project
- Checkpoint accounts for 5% of the grade of the project
- Final submission accounts for the remaining 65 % of the grade of the project
- The minimum final project grade is 35%
- **During the defense, every student must be able to justify the design choices and implementation details of all aspects of the project and why, regardless of who developed or implemented them. This is a course project and all members are expected to understand the entire system. Failure to adequately justify or support the submitted implementation will be treated as an attempt to take credit for work performed by others. In such cases, the student will receive a grade of 0, and disciplinary actions may be taken.**

Technical constraints

- The Entity-Relationship model must be created using **ONDA**.
- The project must be implemented in **Python**.
- Database engine: **PostgreSQL**.
- API testing must be demonstrated with **Postman**.
- The use of ORMs is **not allowed**.
- SQL should use operators taught in class.
- Advanced operators outside the taught scope are allowed only if explicitly justified in the report and argued in the defense.
- **CTE, RANK, ROW_NUMBER or other window functions are not allowed.**
- Data processing (sorting/filtering/aggregation) must be done in SQL whenever possible (not in Python post-processing).

Annex A: Metro Mondego System Context – Functional Description

Metro Mondego is an innovative urban transport company, dedicated to providing an efficient, sustainable and affordable service for mobility in the Coimbra region and its surroundings. With a modern and well-integrated surface metro system, the company offers passengers a convenient and environmentally friendly alternative for their daily journeys.
With several lines of transport, Metro Mondego covers the main urban axes, reducing journey times and promoting accessibility for the entire population. The vehicles are modern, comfortable, and equipped with state-of-the-art technology, guaranteeing pleasant and safe journeys. In addition, the system was designed to be sustainable, using electric fleets and adopting ecological measures that contribute to reducing the city's carbon footprint.
Metro Mondego also stands out for its digital service, which allows users to consult the different transport lines, check timetables in real time and purchase tickets or passes directly through a ticket management system. The system supports the following ticket types: single trip, daily (24h), monthly pass, monthly student pass, and monthly senior pass. The history of ticket prices should be retained over time so that administrators can review the cost of tickets purchased in the past for analytics purposes.
The digital system also offers advantages for companies and institutions, making it possible to purchase corporate and academic passes, as well as special rates for students, the elderly and frequent users.
Committed to excellence and accessibility, Metro Mondego works every day to make urban mobility more fluid and sustainable, guaranteeing a quality service that improves the experience of all passengers. Whether for work, leisure or study, travelling with Metro Mondego is synonymous with efficiency, comfort and respect for the environment.
The aim of this project is to design and implement a PostgreSQL database that supports the system and to develop Python endpoints that both clients and administrators can use as described below. The implementation should follow the skills learnt in class, efficient queries, row and aggregation functions, and triggers, taking advantage of PostgreSQL's features.
Below is a brief description of the functionalities to be implemented, adjusted to the permissions of each type of user.

Fixed Metro Lines

The system must include the following fixed lines (not editable as entities, only operational parameters can be changed). Each line has an ordered list of stations.

1. **Portagem – Hospital** (urban metro)
2. **Portagem – Estação B** (urban metro)
3. **Portagem – Miranda do Corvo – Lousã (Serpins)** (regional service)

Baseline operational schedule

Each line has an ordered list of stations. Each scheduled departure of a line in a given direction at a specific date and time corresponds to a trip, which represents a concrete instance of service with its own departure timestamp and capacity.

- **Line 1** (both directions): start 07:30, every 20 minutes, until 21:00.
- **Line 2** (both directions): start 07:45, every 30 minutes, until 19:00.
- **Line 3 Serpins → Portagem**: start 07:00, every 30 minutes until 09:30; then every 1h30 until 19:00.
- **Line 3 Portagem → Serpins**: start 08:00, every 1h30 until 17:00; then every 30 minutes until 20:00.

Default vehicle capacity: **50** passengers (administrator configurable).

Roles and permissions

Super Administrator (single hard-coded account)

- Full system permissions.

- Creates/grants/revokes Administrator role to users.
- Can perform all Administrator operations.

Administrator

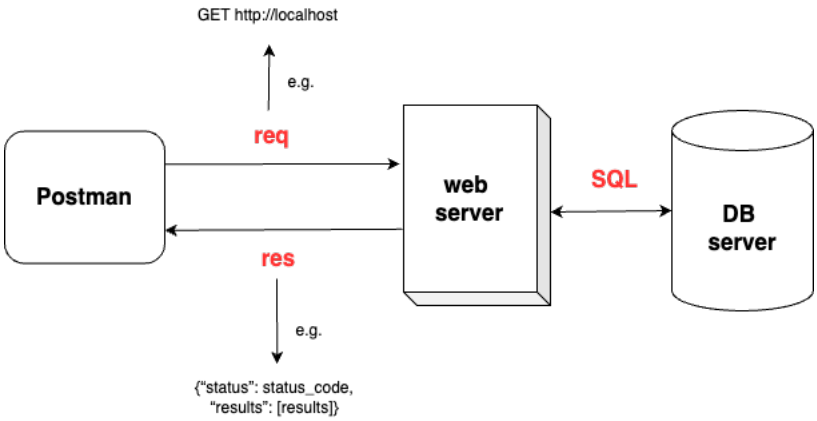
- Manages operations, fares, policies, and communications.
- Accesses reporting/statistics endpoints.
- Can close/reopen lines temporarily (maintenance/events).

Customer

- Registers and manages personal account.
- Maintains wallet balance.
- Consults schedules, buys/uses/cancels tickets/passes.
- Receives and reads alerts.

Technical Description

The figure represents a simplified view of the system to be developed. The system must be made available through a REST API that allows the user to access it through HTTP requests (when content is needed, JSON must be used). As it can be seen, the user interacts with the web server through a REST request/response exchange, and in turn, the web server interacts with the database server through an SQL interface (e.g., psycopg2/psycopg3 in the case of Python). This is one of the most used architectures today and supports many web and mobile applications we use daily. Since the course focuses on the design and operation of a data management system and associated functionalities, the development of web or mobile applications is outside the scope of this work. To use or test the REST API, you must use the REST client Postman (postman.com).



To facilitate the development of the project and focus on the data management system, groups should follow/use the demo code that will be made available. An example of how to test the REST API endpoints using Postman is also provided. In the event of missing details, groups should explicitly identify them in the report.

HTTP works as a request-response protocol. For this work, three main methods might be necessary:

- **GET**: used to request data from a resource
- **POST**: used to send data to create a resource
- **PUT**: used to send data to update a resource
- **DELETE**: used to delete a resource

REST API responses should follow a simple but rigid structure, always returning a status code (this is a very simplified version of how a real REST API would work):

- 200 – success
- 400 – error: bad request (request error)

- 500 – error: internal server error (API error)

If there are errors, they must be returned in *errors*, and if there are data to be returned, they must be returned in *results* (as can be seen in the details of the endpoints that follow).

Functionalities to be developed

The *endpoints* of the REST API should be **strictly followed**. Any details that are not defined should be explained in the report.

When the user starts using the platform, he/she can choose between registering a new account or logging in using an existing account. The following *endpoints* should be used.

***NOTICE**: This is a simplified scenario. In a real application, a secure/encrypted connection would be used to send the credentials (e.g., HTTPS).*

1. **User Authentication.** Login using the *username* and the *password* and receive an *authentication token* (e.g., JSON Web Token (JWT), https://jwt.io/introduction) in case of success. This *token* should be included in the *header* of the remaining requests.

```
req   PUT http://localhost:8080/dbproj/user
      {“username”: username, “password”: password}
res   {“status”: status_code, “errors”: errors (if any occurs), “results”: auth_token (if it
      succeeds)}
```

After the authentication, the user can perform the following operations using the obtained *token* during the user authentication (the token should always be passed in every request, either in the body or in authentication headers). **Some operations are restricted to certain types of users.**

Super Admin functionalities

2. **Add Administrator.** Create a new administrator, inserting the data required by the data model. Take into consideration the individual type and attributes to be consider. Do not forget that different types of users have different attributes (example is provided). Avoid duplicated code. Remember that user details may contain sensitive data. Only Super Administrators can add new administrators (one account should be added by default when creating the database).

```
req   PUT http://localhost:8080/dbproj/register/admin
      {
        "name": "Admin Name",
        "email": "customer@email.pt",
        "password": "secret"
      }
res   {"status": status_code, "errors": errors, "results": {"user_id": id}}
```

Admin functionalities

3. **Add Customer.** Create a new individual, inserting the data required by the data model. Take into consideration the individual type and attributes to be consider. Do not forget that different types of users have different attributes (example is provided). Avoid duplicated code. Remember that user details may contain sensitive data. Only Administrators can add new individuals.

```
req   POST http://localhost:8080/dbproj/register/customer
      {
        "name": "Customer Name",
        "nif": "123456789",
        "phone": "910000000",
        "email": "customer@email.pt",
        "password": "secret"
      }
res   {"status": status_code, "errors": errors, "results": {"user_id": id}}
```

4. **Update line operation settings.** Update a given line with new data.

req PUT http://localhost:8080/dbproj/line_operation/{line_id}

```
{
  "start_time": "07:30:00",
  "end_time": "21:00:00",
  "frequency_minutes": 20,
  "vehicle_capacity": 50
}
```

res

```
{"status": status_code, "errors": errors}
```

5. **Update fare price.** Change the price of the fare (ticket).

req PUT http://localhost:8080/dbproj/fares/{fare_id}

```
{"price": 2.75, "effective_from": "2025-06-01"}
```

res

```
{"status": status_code, "errors": errors}
```

6. **Broadcast notice.** Provide a notice to all customers.

req POST http://localhost:8080/dbproj/notices/broadcast

```
{"title": "Strike Notice", "message": "Possible delays between 10:00 and 13:00"}
```

res

```
{"status": status_code, "errors": errors}
```

7. **Create promotion/discount rule.** Insert a promotion so users can buy cheaper tickets.

req POST http://localhost:8080/dbproj/promotions

```
{
  "name": "School Holidays",
  "line_id": 1,
  "product_type": "daily",
  "discount_percent": 20,
  "start_date": "2025-07-01",
  "end_date": "2025-07-31"
}
```

res

```
{"status": status_code, "errors": errors, "results": {"promotion_id": id}}
```

Customers functionalities

8. **List lines and upcoming departures.** A customer can get all the information regarding all lines and next departure time as well as capacity for the metro.

req GET http://localhost:8080/dbproj/lines_next

res

```
{
  "status": status_code,
  "errors": errors,
  "results": [
    {
      "line_id": 1,
      "line_name": "Portagem-Hospital",
      "origin_terminal": "Portagem",
      "destination_terminal": "Hospital",
      "departure_time": "2026-04-10 08:10:00",
      "estimated_delay_min": 0,
      "available_capacity": 18
    },
    {
      "line_id": 1,
      "line_name": "Portagem-Hospital",
      "origin_terminal": "Hospital",
      "destination_terminal": "Portagem",
      "departure_time": "2026-04-10 08:20:00",
      "estimated_delay_min": 0,
```

```
            "available_capacity": 10
          },
          …
        ]
      }
```

9. **Add wallet funds.** A customer can add funds to his/her wallet.

```
req   POST http://localhost:8080/dbproj/wallet/topup
      {"amount": 20.00, "payment_method": "card"}
res   {"status": status_code, "errors": errors, "results": {"new_balance": 48.30}}
```

10. **Purchase ticket/pass.** Buy a single trip fare or a montly pass. Use triggers to automatically deduct the cost from the customer funds.

```
req   POST http://localhost:8080/dbproj/purchase
      {
        "line_id": 2,
        "product_type": "single_trip",
        "travel_date": "2025-04-12"
      }
res   {"status": status_code, "errors": errors, "results": {"purchase_id": id, "final_price":
      1.50}}
```

11. **Validate/use ticket.** Register a trip in a given ticket. Validate that the ticket is valid for that line.

```
req   POST   http://localhost:8080/dbproj/ticket/use/{ticket_id}
      {"used_at": "2025-04-12 08:20:00", "station_id": 7}
res   {"status": status_code, "errors": errors}
```

Analytics functionalities (Administrators)

12. **Peak and Low demand periods.** List all lines with the time interval that accounts for the higher and lower passengers travelling for each line. Just one *SQL* query should be used to obtain the information.

```
req   GET http://localhost:8080/dbproj/report/demand
res   {
        "status": status_code,
        "errors": errors,
        "results": [
          {"line_id": 1, "time_slot": "08:00-08:59", "validations": 1430},
          {"line_id": 2, "time_slot": "13:00-13:59", "validations": 2000},
          ...
          {"line_id": 1, "time_slot": "19:00-19:59", "validations": 30},
          ...
        ]
      }
```

13. **Top spenders by line**. For each line, list the customer(s) with the highest **spending** over the last 30 days. Just one *SQL* query should be used to obtain the information.

```
req   GET  http://localhost:8080/dbproj/report/top_spenders
res   {
        "status": status_code,
        "errors": null,
        "results": [
          {
            "line_id": 1,
            "customer_id": 17,
```

```
            "customer_name": "Ana Silva",
            "spent": 132.50
          }
        ],
      …
      }
```

14. **Generate a monthly report**. For each line and month, show how many unique customers (>=1 ticket validation) used the service and how many were repeat users (>= 2 ticket validation). Just one *SQL* query should be used to obtain the information.

```
req   GET  http://localhost:8080/dbproj/report/monthly
res   {
        "status": status_code,
        "errors": null,
        "results": [
          {
            "line_id": 2,
            "month": 1,
            "active_customers": 1840,
            "repeat_customers": 620
          },
          …
        ]
      }
```

FINAL REMARKS

- As it is defined in some *endpoints*, the implemented solution should obtain the information using **a single SQL *query*** on the server side (subqueries are allowed). This does not include potential steps to validate the user authentication. **Nevertheless**, in case you cannot solve it with a single query, it is preferable to use more *queries* than not implementing the *endpoint*.
- The data processing (e.g., order, restrictions) should be done in the *queries* whenever possible (and not in code).
- Database systems are designed to operate under intensive concurrent execution. Proper transaction design, identification of potential concurrency conflicts, and the implementation of appropriate mitigation strategies are expected. Concurrency control and data integrity preservation will be explicitly evaluated.